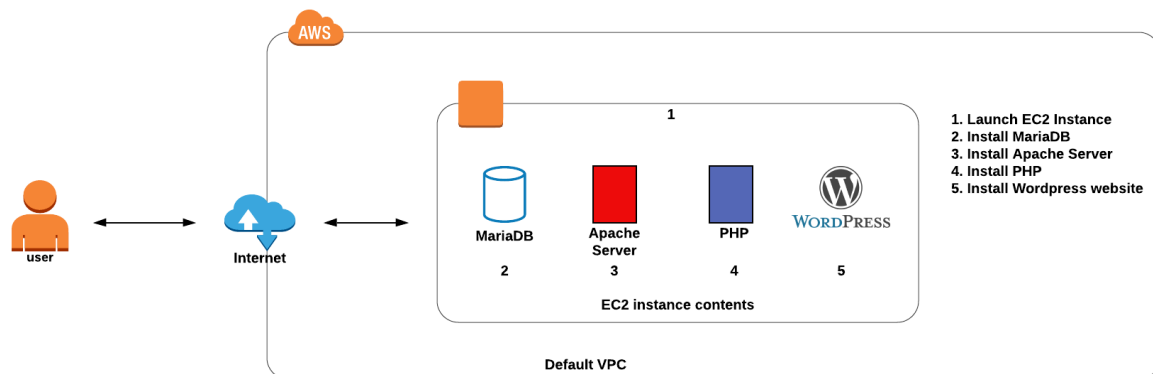


Build Your Own Wordpress Website Using AWS Console

Lab Details

1. This lab walks you through the step by step procedure of launching an EC2 instance and then installing WordPress on it. You will be performing these actions via SSH on your terminal. Once Wordpress is installed, you will log into the Wordpress website.
2. Duration: **60 minutes**
3. AWS Region: **US East (N. Virginia) us-east-1**

Architetcure Diagram



Task Details

1. Launching Lab Environment
2. Launch an EC2 instance.
3. SSH into the EC2 instance.
4. Run a test page in your browser.
5. Setting up permissions and LAMP server
6. Database Server Security Details
7. Optional installation for handling the administration of MySQL over the Web with phpMyAdmin
8. Install WordPress
9. Create and edit the WordPress Config file
10. To provide permission for WordPress to use permalinks

11. Run the WordPress installation script
12. Validation of the lab

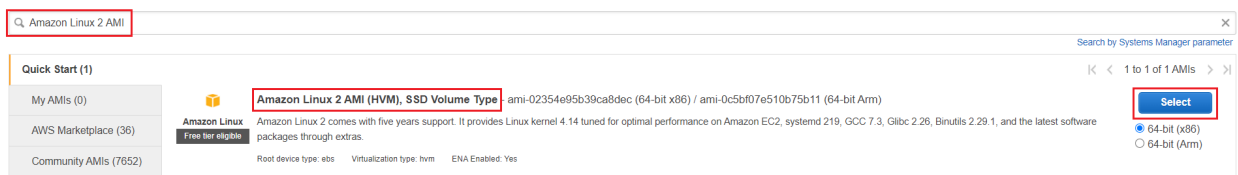
Lab Steps

Task 1: Launching an instance

1. Make sure you are in the **N.Virginia** Region.
2. Navigate to **EC2** by clicking on the **Services** menu in the top, then click on **EC2** in the **Compute** section.
3. **Choose an Amazon Machine Image (AMI):** Search for **Amazon Linux 2 AMI** in the search box and click on the **select** button.
4. Navigate to **Instances** from the left side menu and click on **Launch Instances** button.

Step 1: Choose an Amazon Machine Image (AMI)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. You can select an AMI provided by AWS, our user community, or the AWS Marketplace, or you can select one of your own AMIs.



5. **Choose an Instance Type:** Select **t2.micro** and then click on **Next: Configure Instance Details**
6. **Configure Instance Details:** No need to change anything in this step, go to the next step by clicking **Next: Add Storage**
7. **Add Storage:** No need to change anything in this step, go to the next page by clicking **Next: Add Tags**
8. **Add Tags:** Click on **Add Tag**
 - Key : Enter **Name**
 - Value : Enter **WordPressInstance**
 - Click on **Next: Configure Security Group**
10. **Configure Security Group:**
 - To add **SSH**:

- Choose Type: SSH
- Source: Custom (Allow specific IP address) or Anywhere (From ALL IP addresses accessible).

- For HTTP:

- Click on Add Rule
- Choose Type: HTTP
- Source: Custom (Allow specific IP address) or Anywhere (From ALL IP addresses accessible).

- For HTTPS:

- Click on Add Rule
- Choose Type: HTTPS
- Source: Custom (Allow specific IP address) or Anywhere (From ALL IP addresses accessible).

- After that, click on Review and Launch

11. **Review and Launch:** Review all of your select settings and click on Launch.

12. **Key Pair :** Create a new key Pair and click on Download Key Pair . After that,

click on Launch Instances . Make sure you keep the keypair in a known location as we need it to SSH into the instance later.

13. **Launch Status:** Your instance is now launching. Click on the instance ID and wait for complete initialization of the instance (until the status changes to running).

Instances (1) Info		Refresh	Connect	Instance state ▼	Actions ▼
Filter instances					
☐	Name ▼	Instance ID	Instance state ▼	Instance type ▼	Status check
☐	WordPressInst...	i-0cb2d8d02aa4db659	Running	t2.micro	-

14. Note the sample IPv4 Public IP Address of the EC2 instance. A sample is shown in the below screenshot:

Instance: i-0cb2d8d02aa4db659 (WordPressInstance)

×

Details

Security

Networking

Storage

Status checks

Monitoring

Tags

▼ Instance summary Info

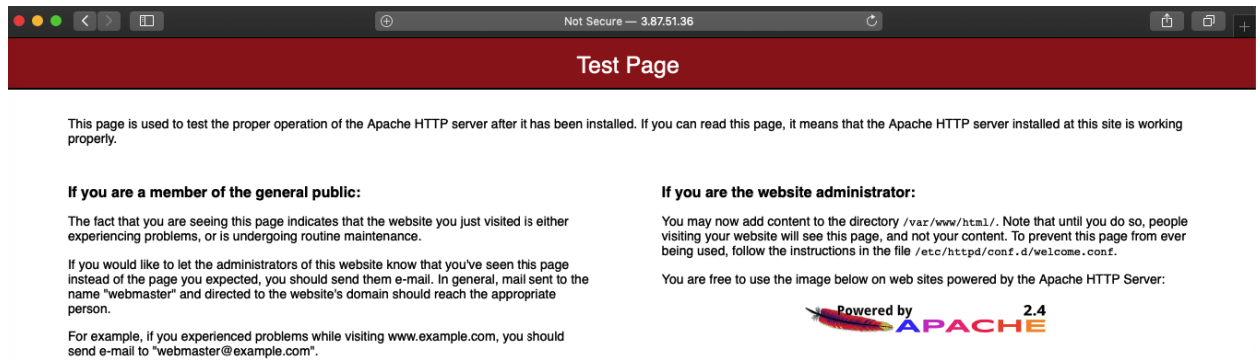
Instance ID i-0cb2d8d02aa4db659 (WordPressInstance)	Public IPv4 address 35.175.227.43 open address	Private IPv4 addresses 172.31.40.54
IPv6 address -	Instance state Running	Public IPv4 DNS ec2-35-175-227-43.compute-1.amazonaws.com open address

Task 2: SSH into the EC2 Instance

1. To SSH into the instance, please follow the steps in [SSH into EC2 Instance](#).

Task 3: Run a test page in your browser

1. To ensure that all programs are up to date, run below command:
 - `sudo yum update -y`
2. Next step is to get the latest versions of MariaDB (a community-developed fork of MySQL) and PHP. Run the following commands to install them both:
 - `sudo amazon-linux-extras install -y lamp-mariadb10.2-php7.2 php7.2`
3. Now lets install Apache server and MariaDB.
 - `sudo yum install -y httpd mariadb-server`
4. Start the Apache Server
 - `sudo systemctl start httpd`
5. We can also make the apache server start automatically every time we boot the instance with following command:
 - `sudo systemctl enable httpd`
6. Test whether Apache Server enabled or not with below command:
 - `sudo systemctl is-enabled httpd`
7. Now it's time to test whether the sample test page of the apache server is running or not.
 - Copy your Public IPv4 address and enter in your browser and hit enter. If you see the below test page, it means apache server is successfully installed.

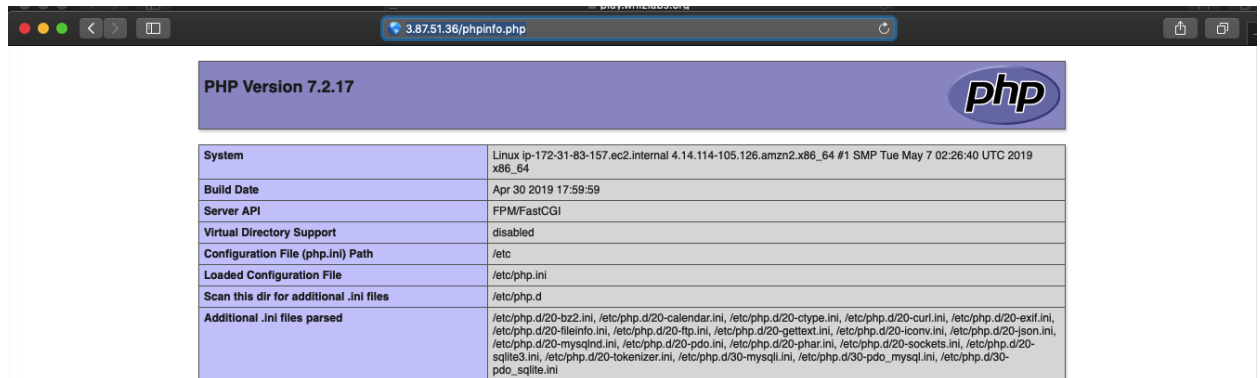


- If the test page is not opening, then you have made some mistake while installing and starting the apache server. Please check the above steps properly and do it again.

Task 4: Setting up permissions and LAMP server

1. The next steps are to set file permissions.
2. Add user (ec2-user) to apache group
 - `sudo usermod -a -G apache ec2-user`
3. Log out and log back in to verify the membership of the new group
 - Execute the logout command or close the window:
 - `exit`
 - SSH into the instance again
 - To verify membership enter the below command:
 - `groups`
4. Now to change the group ownership of `/var/www` and the content inside of it to the apache group:
 - `sudo chown -R ec2-user:apache /var/www`
5. In order to add group write permissions, to set the group ID on future subdirectories and also change the directory permissions of `/var/www` and its subdirectories:
 - `sudo chmod 2775 /var/www && find /var/www -type d -exec sudo chmod 2775 {} \;`
6. To add group write permissions, recursively change the file permissions of `/var/www` and its subdirectories:
 - `find /var/www -type f -exec sudo chmod 0664 {} \;`
7. Now, the EC2 user (and any future members of the apache group) can add, delete, and edit files in the root of the Apache document. This allows you to add content like a static website or a PHP application.
8. Let's test the LAMP server now.

9. We will create a PHP file at the root of the Apache document. This will become our home page to test if everything is working as expected.
 - `echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php`
10. To test the example PHP page, open a tab in the browser and enter `"/phpinfo.php"` after the Public IPv4 address as shown in the below image.



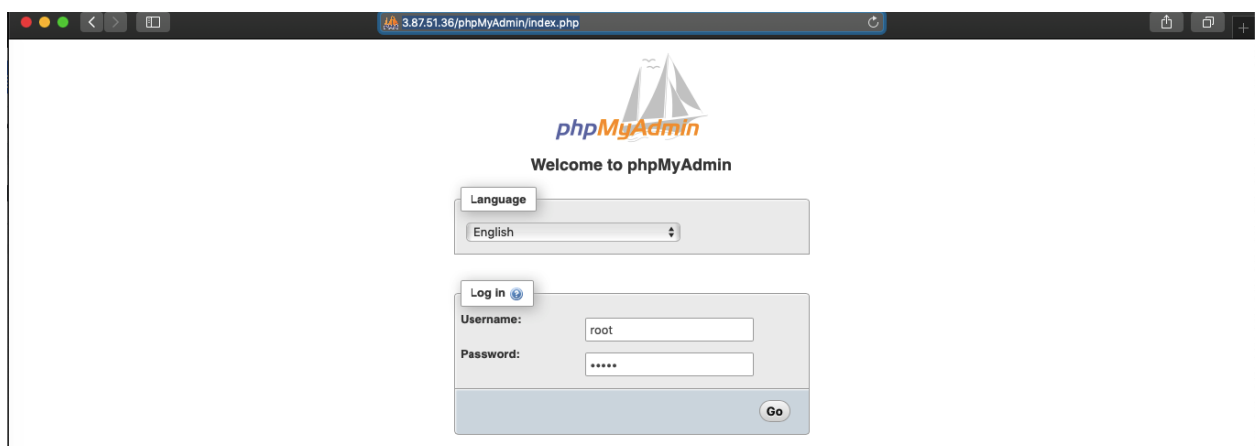
11. Now that we have tested the example site, we can delete the `phpinfo.php` file.
12. Let's delete the `phpinfo.php` file. We just used it to test, for security reasons, these details should not be available on the internet.
 - `rm /var/www/html/phpinfo.php`

Task 5: Database Server Security Details

1. To start the MariaDB server:
 - `sudo systemctl start mariadb`
2. To install MySQL, run the `mysql_secure_installation`.
 - `sudo mysql_secure_installation`
 - When prompted for a password, type a password for the root account. (by default, the root account does not have a password). Press Enter when finished.
 - Type **Y** to set the password, and retype the password to confirm it.
 - **Note:** Make a note of the new password which you set as it will be used in future.
 - Type **Y** to remove the anonymous user accounts.
 - Type **Y** to disable the remote root login.
 - Type **Y** to remove the test database.
 - Type **Y** to reload the privilege tables and save your changes.
3. Optional: We can also make the MariaDB server start on every boot, . If you would like to enable this, type the following command:
 - `sudo systemctl enable mariadb`

Task 6: Optional installation for handling the administration of MySQL over the Web with phpMyAdmin

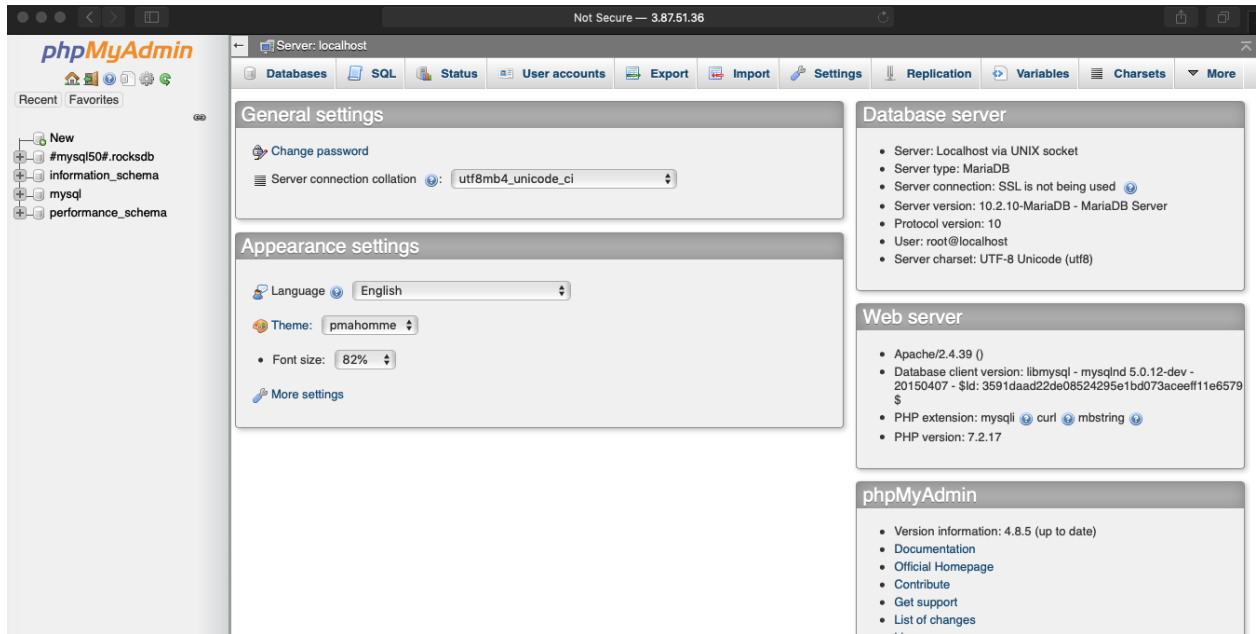
1. Let's install the required dependencies:
 - `sudo yum install php-mbstring php-xml -y`
2. Restart the Apache Server:
 - `sudo systemctl restart httpd`
3. We also need to restart php-fpm.
 - `sudo systemctl restart php-fpm`
4. Navigate to the root of the Apache document at `/var/www/html`.
 - `cd /var/www/html`
5. We have to select a source package for the latest phpMyAdmin release from <https://www.phpmyadmin.net/downloads>. To download the file directly to your instance, copy the link and paste it into a wget command as shown below:
 - `wget https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz`
6. Create a phpMyAdmin folder and extract the package into it with the following command:
 - `mkdir phpMyAdmin && tar -xvzf phpMyAdmin-latest-all-languages.tar.gz -C phpMyAdmin --strip-components 1`
7. Delete the phpMyAdmin-latest-all-languages.tar.gz tarball.
 - `rm phpMyAdmin-latest-all-languages.tar.gz`
8. Run the following command to make sure MySQL is running:
 - `sudo systemctl start mariadb`
9. In a web browser, type the below URL to see the php admin page:
 - `http://yourIPAddress/phpMyAdmin`
10. Log into your phpMyAdmin installation with your root user name and the MySQL root password we created earlier.



11. **Note(Optional):** If you see an error saying that xml is not enabled then only run the below command.

- `sudo apt install php-xml`
- `sudo apt-get install php-mbstring`

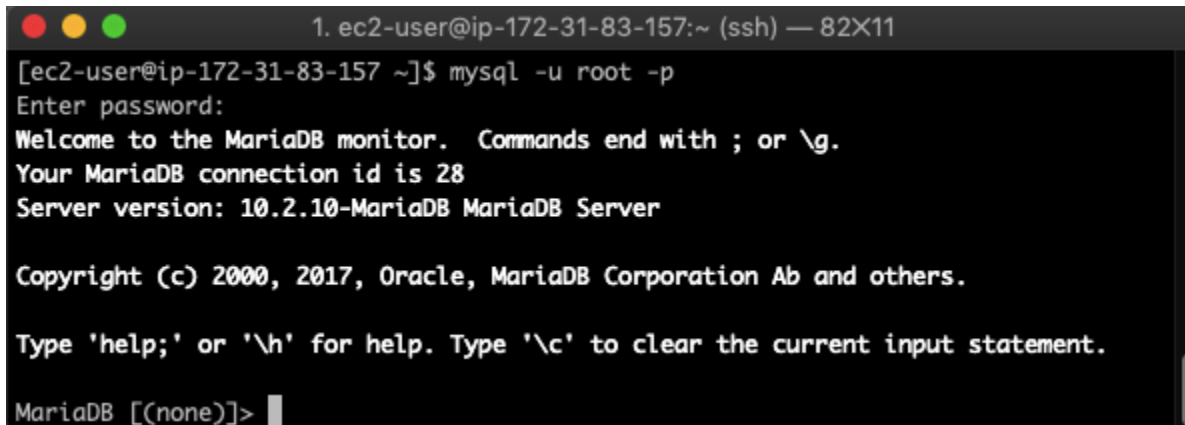
11. Once you login with root credentials, it will look like this:



Task 7: Install WordPress

1. Go back to Terminal and SSH into the instance. (Use the **cd** command to go to the parent directory if you are in var/www/html)
2. Download and unzip the WordPress installation package. To download the latest WordPress installation package with wget, use the following command to always download the latest release:
 - `wget https://wordpress.org/latest.tar.gz`
3. Unzip and unarchive the installation package. The installation folder is unzipped to a folder called wordpress.
 - `tar -xzf latest.tar.gz`
4. Lets create a **database user** and a **database** for the WordPress installation.
5. WordPress needs to store information such as blog posts and user comments in a database. The following procedure helps us to create the blog's database and a user that is authorized to read and save information to it.
6. Start the database server to make sure MySQL is running.
 - `sudo systemctl start mariadb`
7. Log into the database server as the root user. Enter the database root password when prompted.

- `mysql -u root -p`



```
1. ec2-user@ip-172-31-83-157:~ (ssh) — 82X11
[ec2-user@ip-172-31-83-157 ~]$ mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 28
Server version: 10.2.10-MariaDB MariaDB Server

Copyright (c) 2000, 2017, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

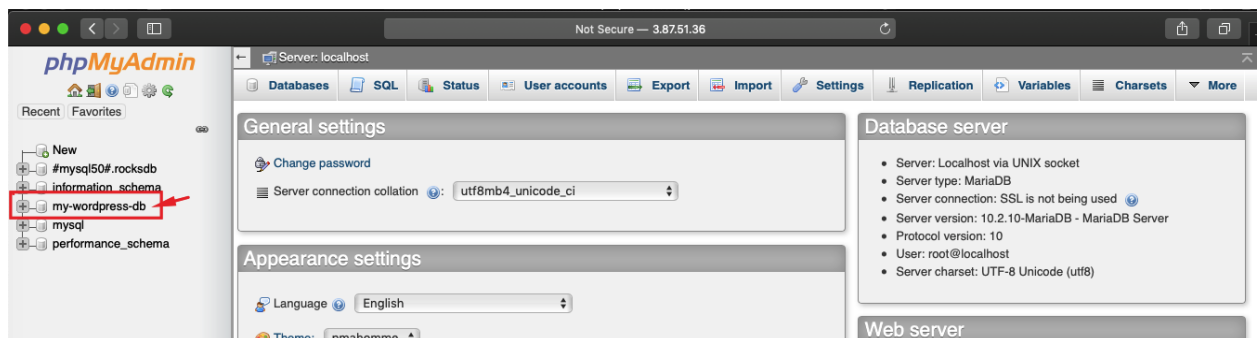
MariaDB [(none)]>
```

8. We will create a **user** and **password** for the MySQL database. WordPress uses these values to communicate with the MySQL database. Enter the following to create a unique **username** and **password**.

- `CREATE USER 'whizlabs-wordpress-user'@'localhost' IDENTIFIED BY 'some_strong_password';`
- **Note:** Make sure to note the username and password as it will be used in the future.

9. Next, we'll create a database. Make sure you give the database a descriptive, meaningful name (such as wordpress-db).

- `CREATE DATABASE `my-wordpress-db` ;`
- Note: The punctuation marks surrounding the database name in the command below are called backticks. The backtick (`) key is usually located above the Tab key on a standard keyboard.
- Make sure to note the database name.
- You can see the created database in phpmyadmin
 - <http://YourIPAddress/phpMyAdmin/index.php>



10. We have to grant full privileges for the database to the WordPress user we created earlier.

- `GRANT ALL PRIVILEGES ON `my-wordpress-db`.* TO "whizlabs-wordpress-user"@"localhost";`
11. We have to flush the database privileges to pick up all of your changes.
 - `FLUSH PRIVILEGES;`
 12. Lastly we'll exit the mysql client.
 - `exit`

Task 8: Create and edit the WordPress Config file

1. The WordPress installation folder contains a sample configuration file called `wp-config-sample.php`. Lets copy this file and edit it to fit our specific configuration.
2. Copy the contents of the `wp-config-sample.php` file to a new file named `wp-config.php`. This creates a new configuration file and keeps the original sample file intact as a backup.
 - `cp wordpress/wp-config-sample.php wordpress/wp-config.php`
3. We can edit the **`wp-config.php`** file with a text editor named `nano` . To open the new config file in `nano`:
 - `nano wordpress/wp-config.php`
4. Find the line that defines **`DB_NAME`** and change **`database_name_here`** to the database name we created. This will [create a database user and database for the WordPress installation](#):
 - `define('DB_NAME', 'my-wordpress-db');`
5. Next, let's find the line that defines **`DB_USER`** and change **`username_here`** to the database user we created. This will [create a database user and database for your WordPress installation](#).
 - `define('DB_USER', 'whizlabs-wordpress-user');`
6. Find the line that defines **`DB_PASSWORD`** and change **`password_here`** to the strong password that we created. This will [create a database user and database for your WordPress installation](#).
 - `define('DB_PASSWORD', 'some_strong_password');`

```
1. ec2-user@ip-172-31-83-157:~ (ssh)
GNU nano 2.9.8                               wordpress/wp-config.php          Modified

* * ABSPATH
*
* @link https://codex.wordpress.org/Editing_wp-config.php
*
* @package WordPress
*/

// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'my-wordpress-db' );

/** MySQL database username */
define( 'DB_USER', 'whizlabs-wordpress-user' );

/** MySQL database password */
define( 'DB_PASSWORD', 'some_strong_password' );

/** MySQL hostname */
define( 'DB_HOST', 'localhost' );

/** Database Charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The Database Collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

AG Get Help      AQ Write Out    AW Where Is     AK Cut Text     AJ Justify      AC Cur Pos      MU Undo         MA Mark Text
AX Exit          AR Read File   AN Replace      AU Uncut Text  AT To Spell     AL Go To Line  ME Redo         MG Copy Text
```

7. Let's Navigate to the section called **Authentication Unique Keys and Salts**. These KEY and SALT values provide a layer of encryption to the browser cookies that WordPress users store on their local machines. Basically, adding long, random values here makes your site more secure.
 - Go to <https://api.wordpress.org/secret-key/1.1/salt/> to randomly generate a set of key values that you can copy and paste into your **wp-config.php** file.
 - **Windows Users:** To paste text into a PuTTY terminal, place the cursor where you want to paste the text and right-click your mouse inside the PuTTY terminal.
8. The values below are samples; use the values which you got while editing.
 - `define('AUTH_KEY', ':_F_D>?2Or4U}r+Q|UMZB$8;*TGn}{bfl!tqv.;]XfMv&L]a3Jq[+xGHjrlt)CH|?');`
 - `define('SECURE_AUTH_KEY', 's~{qL!El|pF_0~{%Ydg^)NRfvf8hj$W|XRt5-%jT*.bfcG6t4|8+&}aNT2FeEdH');`
 - `define('LOGGED_IN_KEY', 'Qn,NHPZWg=;[>=85>g_M4V-+/vi-`z0`+eu|:2n^`v[fj6opp0;NA(Y-D#fzl%');`
 - `define('NONCE_KEY', 'R`dl?S_o!PK:uT:tK<eRLN5Pc(EE%>(E0L({0T@qIDvyr)|k@z:bsvEVk7BTle[6');`

- `define('AUTH_SALT', '$,sGxt,OZV:Gil}e8$=Gh~6pP*o ,>.C; M|TwI@#]uadg1gA&.$d>bd!2mdC}w/');`
- `define('SECURE_AUTH_SALT', '&Af(e4*~ 7D-D*j3=ne>IGmD4]1}/Li>TrdD[wSG9L7wQ# uPi,r*RDPXTMB/}xB');`
- `define('LOGGED_IN_SALT', '73V21RXiM#O@yo U@wVwYZTy-A5_c|+bC)w(KPn1[b 2`$<0 9e2ivu:gnH3YM&~');`
- `define('NONCE_SALT', 'SdiBLts,ma[xf-b-moL*Lh7S>|W,W8|CMU?w)mH)+3QO|7@eeAMfDt_R 2[C)=8MT');`
- **Save** the file and exit your text editor. Press **ctrl+o** and enter to save. Press **ctrl+x** to exit.

```

1. ec2-user@ip-172-31-83-157:~ (ssh)
GNU nano 2.9.8                               wordpress/wp-config.php                               Modified

/**#@+
 * Authentication Unique Keys and Salts.
 *
 * Change these to different unique phrases!
 * You can generate these using the {link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}
 * You can change these at any point in time to invalidate all existing cookies. This will force all users to have to log in
 *
 * @since 2.6.0
 */
define( 'AUTH_KEY',         'F_D>?20r4U}r+QIUMZB$8;*Tgn}[bf!tqv.;]X fWwL]a3Jq[+xGHjr!t)CHI?' );
define( 'SECURE_AUTH_KEY', 's~{qL!EI|pF_0~{Ydg^}NRfvf8hj$W|XRt5-%jT*.bfcG6t4|8+&}aNT2Fe EdH' );
define( 'LOGGED_IN_KEY',   'Qn,NHPZWg-;[>=85>g_M4V-+/vi-`z0'+eul:2n^`v[fj6o$P$P0;NA(Y-D#fzL%' );
define( 'NONCE_KEY',       'R`dL?S_o!PK:uT:tK<eRLN5Pc(EE%>(E0L({0TeqLbvyr)Ik@z:bsvEVk7BTle[6' );
define( 'AUTH_SALT',       '$,sGxt,OZV:Gil}e8$=Gh~6pP*o ,>.C; M|TwI@#]uadg1gA&.$d>bd!2mdC}w/' );
define( 'SECURE_AUTH_SALT', '&Af(e4*~ 7D-D*j3=ne>IGmD4]1}/Li>TrdD[wSG9L7wQ# uPi,r*RDPXTMB/}xB' );
define( 'LOGGED_IN_SALT',  '73V21RXiM#O@yo U@wVwYZTy-A5_c|+bC)w(KPn1[b 2`$<0 9e2ivu:gnH3YM&~' );
define( 'NONCE_SALT',      'SdiBLts,ma[xf-b-moL*Lh7S>|W,W8|CMU?w)mH)+3QO|7@eeAMfDt_R2[C)=8MT' );

/**#@-*/

/**
 * WordPress Database Table prefix.
 *
 * You can have multiple installations in one database if you give each
 * a unique prefix. Only numbers, letters, and underscores please!
 */

```

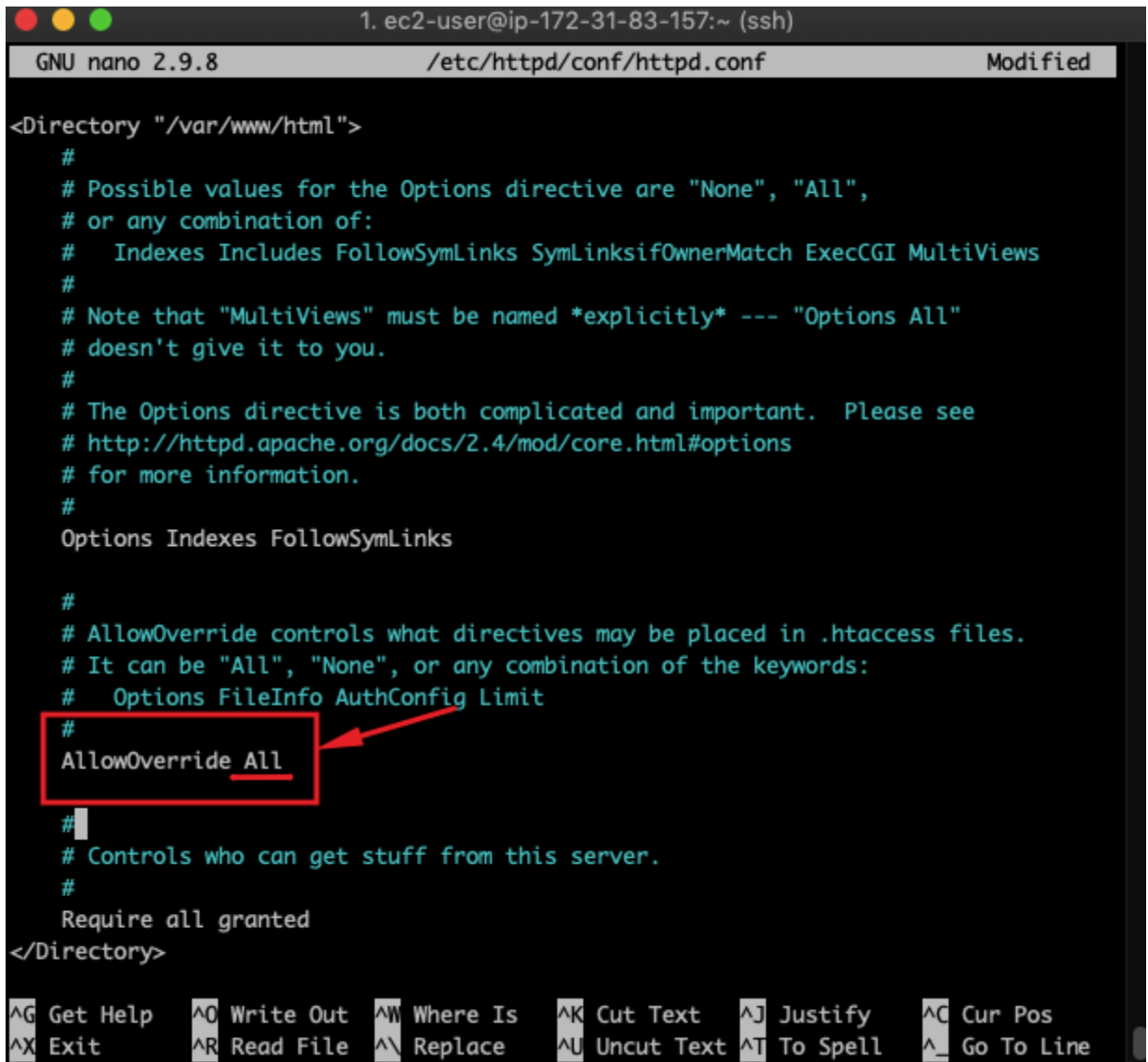
To save changes

Get Help Write Out Where Is Cut Text Justify Cur Pos Undo Mark Text
Exit Read File Replace Uncut Text To Spell Go To Line Redo Copy Text

- Now that we have unzipped the installation folder, created a MySQL database and user, and customized the WordPress configuration file, we are ready to copy the installation files to the root of the web server so we can run the installation script that completes the installation.
- Lets create a folder by name mywordpresswebsite under which we will copy all the wordpress related files:
 - `mkdir /var/www/html/mywordpresswebssite`
- Copy the files to mywordpresswebssite:
 - `cp -r wordpress/* /var/www/html/mywordpresswebssite/`

Task 9: To provide permission for WordPress to use permalinks

1. Permissions on the Apache .htaccess files are needed for wordpress to use permalink however, this is not enabled by default on Amazon Linux. We need to allow all overrides in the Apache document root. To do so, we need to edit the httpd.conf file
 - `sudo nano /etc/httpd/conf/httpd.conf`
2. Find the section that starts with **<Directory "/var/www/html">**.
3. Change the AllowOverride None line in the above section to read AllowOverride All.
 - `AllowOverride All`



```
1. ec2-user@ip-172-31-83-157:~ (ssh)
GNU nano 2.9.8 /etc/httpd/conf/httpd.conf Modified

<Directory "/var/www/html">
#
# Possible values for the Options directive are "None", "All",
# or any combination of:
#   Indexes Includes FollowSymLinks SymLinksifOwnerMatch ExecCGI MultiViews
#
# Note that "MultiViews" must be named *explicitly* --- "Options All"
# doesn't give it to you.
#
# The Options directive is both complicated and important. Please see
# http://httpd.apache.org/docs/2.4/mod/core.html#options
# for more information.
#
Options Indexes FollowSymLinks

#
# AllowOverride controls what directives may be placed in .htaccess files.
# It can be "All", "None", or any combination of the keywords:
#   Options FileInfo AuthConfig Limit
#
AllowOverride All

#
# Controls who can get stuff from this server.
#
Require all granted
</Directory>

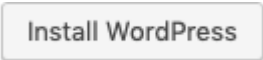
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Uncut Text ^T To Spell  ^_ Go To Line
```

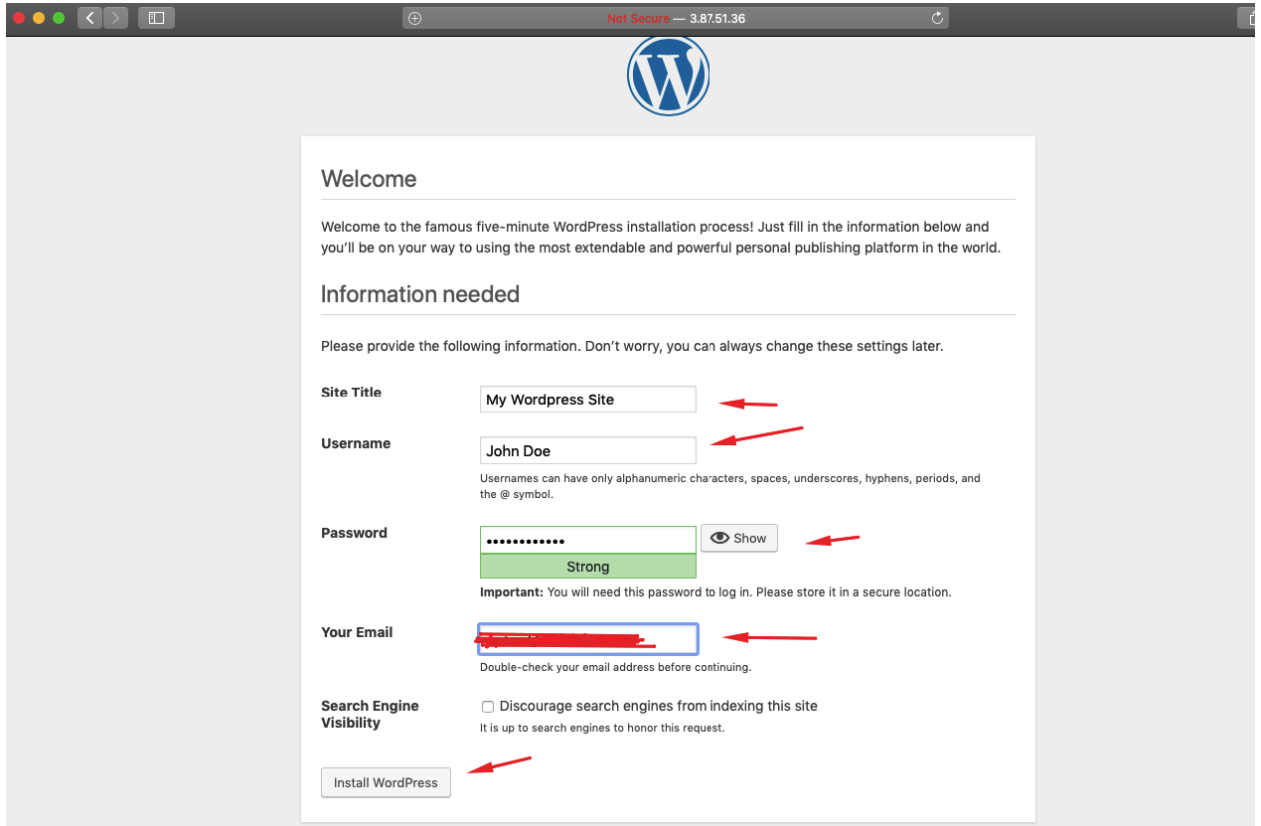
Note: There are multiple AllowOverride lines in this file; be sure you change the line in the **<Directory "/var/www/html">** section.

4. Save the file and exit your text editor. **Press ctrl+o and enter** to save. Press **ctrl+x** to exit.
5. Next we will provide file permissions for the Apache web server. Apply the following group memberships and permissions (as described in **Setting up permissions and LAMP server**).
6. We have to grant file ownership of **/var/www** and its contents to the apache user:
 - `sudo chown -R apache /var/www`
7. Next, grant group ownership of **/var/www** and its contents to the Apache group:
 - `sudo chgrp -R apache /var/www`
8. We have to change the directory permissions of **/var/www** and its subdirectories to add group write permissions and to set the group ID on future subdirectories.
 - `sudo chmod 2775 /var/www`
 - `find /var/www -type d -exec sudo chmod 2775 {} \;`
 - **Note:** Wait until the command is executed and the command prompt returns.
9. Recursively change the file permissions of **/var/www** and its subdirectories to add group write permissions.
 - `find /var/www -type f -exec sudo chmod 0664 {} \;`
 - **Note:** Wait until the command is executed and the command prompt returns.
10. Restart the Apache web server to pick up the new group and permissions.
 - `sudo systemctl restart httpd`

Task 10: Run the WordPress installation script

1. Now we are finally ready to install WordPress.
2. We will use the **systemctl** command to ensure that the **httpd** and **database services** start at every system boot, if not already done.
 - `sudo systemctl enable httpd && sudo systemctl enable mariadb`
3. We have to verify that the database server is running.
 - `sudo systemctl status mariadb`
 - If the database service is not running, start it.
 - `sudo systemctl start mariadb`
4. Also verify that your Apache web server (httpd) is running.
 - `sudo systemctl status httpd`
 - If the httpd service is not running, start it.
 - `sudo systemctl start httpd`
5. In a web browser, type the below URL to see the php admin page:
 - `http://YourIPAddress/mywordpresswebsite`

6. You should see the WordPress installation script Page. Provide the information required by the WordPress installation. Choose  to complete the installation.



Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Don't worry, you can always change these settings later.

Site Title

Username

Username can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password

Strong

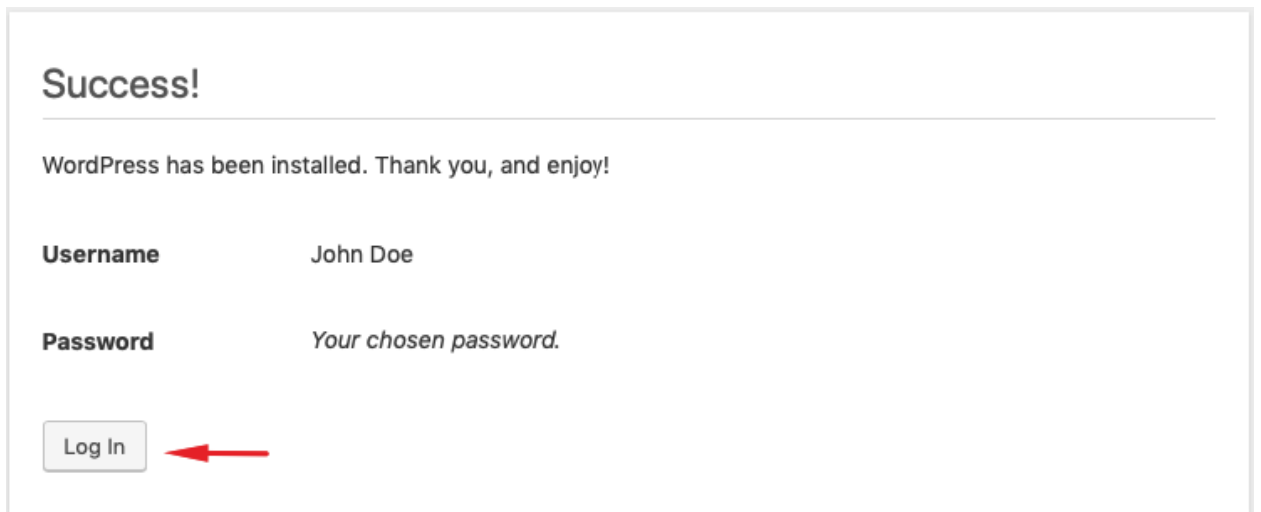
Important: You will need this password to log in. Please store it in a secure location.

Your Email

Double-check your email address before continuing.

Search Engine Visibility ☐ Discourage search engines from indexing this site
It is up to search engines to honor this request.

7. Once the wordpress is installed, click on the  button on the next page.



Success!

WordPress has been installed. Thank you, and enjoy!

Username John Doe

Password Your chosen password.

8. Enter Username as **John Doe** and the login password as the same password you set on the previous page. Click on 

Username or Email Address

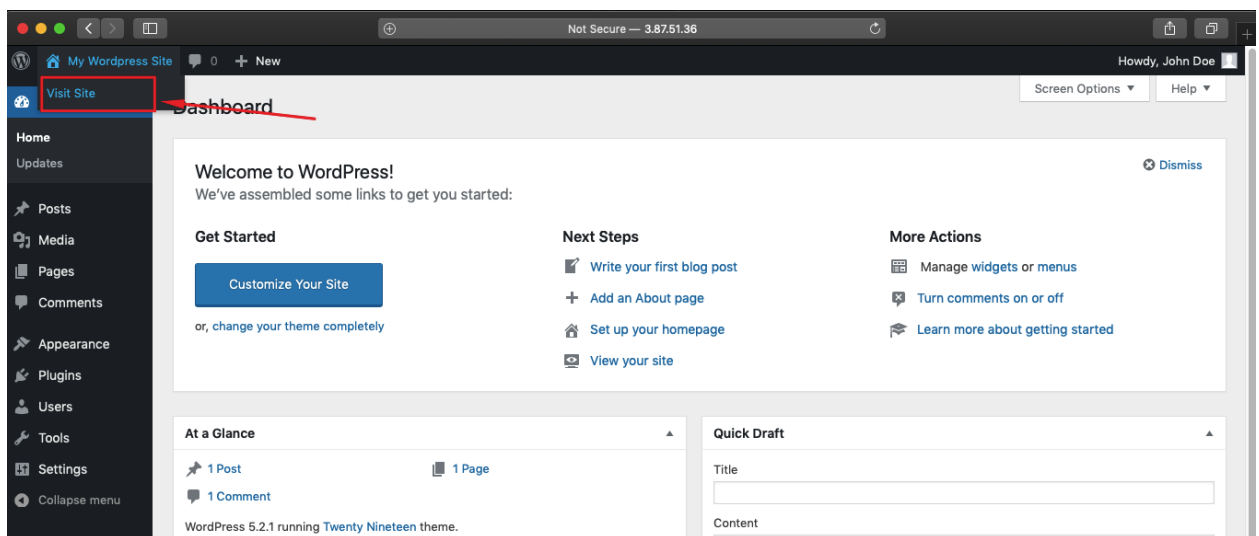
John Doe

Password

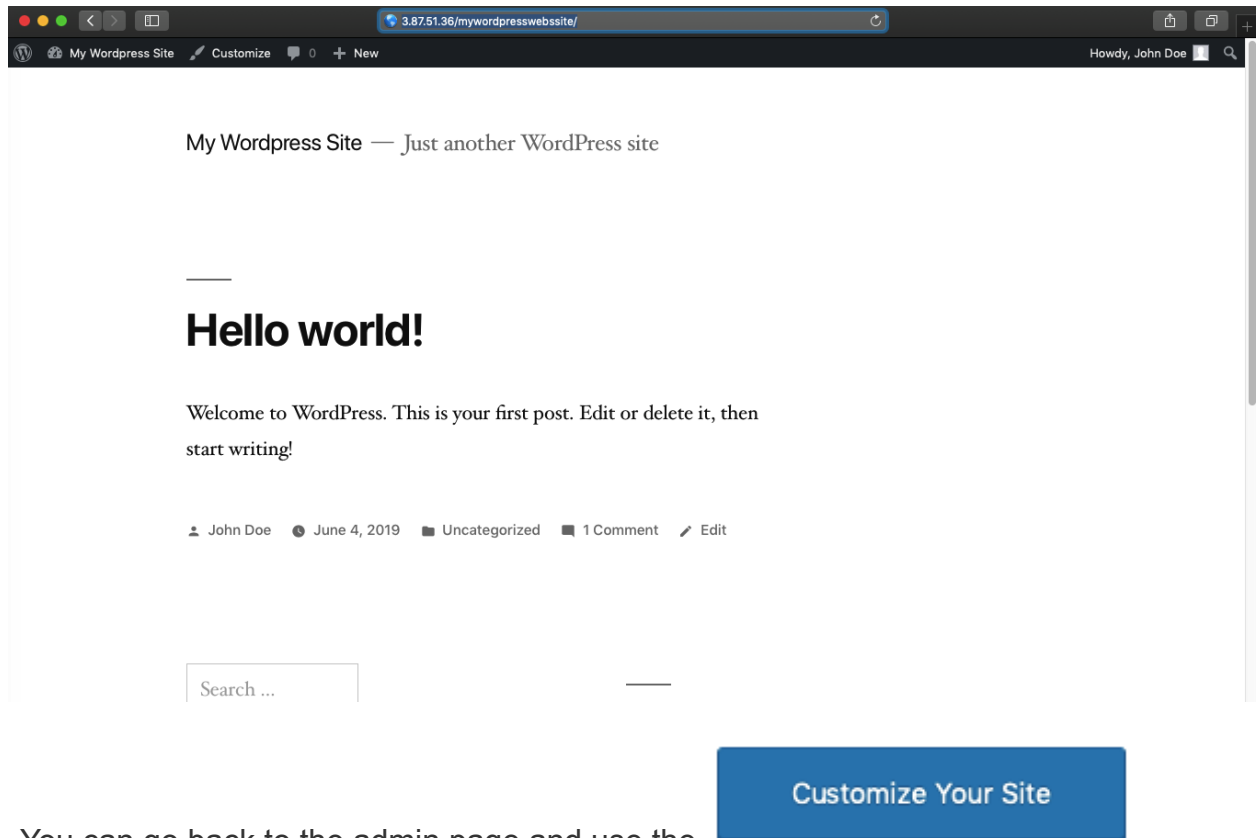
●●●●●●●●●●

☐ Remember Me 

9. You will see the admin page of Wordpress. Click on  to see the website.



10. Here you will see the website we created!



11. You can go back to the admin page and use the button to customize the website details as you like.
12. This completes the lab for creating your own Wordpress website.

Completion and Conclusion

1. You have successfully created and launched an Amazon EC2 Instance.
2. You have successfully installed Wordpress.
3. You have successfully logged into the Wordpress site.
4. You have successfully visited a sample website created by Wordpress.